

雙變量函數的繪圖 & 多變量函數的參數估計與極值計算專題研究

學號：411078060

姓名：黃琮茗

作品目標:這次作品藉由混合常態的模擬來初步了解非特定分配樣本的估計，選出不同的兩組(雙峰/單峰)，讓我們可以比較是否因為兩個常態較貼合(看似一個)，就需要更多的樣本去解析說是兩個。本例題運用到自己做演算法與 GaussianMixture 兩個去與 true 進行對比，在這例題中，可以去比較兩者差異，進而進行更多深入的分析。第二題則是將 MLE 問題透過課堂中第一次學習的立體圖與等高線圖對值得分布，或是極大值去做了解。

研究一：混合常態參數估計 Normal Mixture

- 1.自行設定資料生成的參數 $\Omega = \{\pi_1, \mu_1, \sigma_1^2, \mu_2, \sigma_2^2\}$ ，設計兩組情況：一組之 μ_1, μ_2 較接近（視覺上好像只有一組），另一組分開較遠些（視覺上看出兩個常態混合）。以下圖一可做為參考。
- 2.總樣本數 $n = 50, 100, 300, 500, 1000, 10000$ 。藉以評估樣本數大小對估計值的影響。
- 3.繪製執行結果，含資料的直方圖、真實的混合常態 PDF 與估計的混合常態 PDF。因每次執行的結果會因為樣本資料之不同而不同，選擇其中某一次即可，如圖二。
- 4.(Optional)混合常態的估計問題已經有一段歷史了，不難想像早已被寫成套件，譬如 `sklearn.mixture.GaussianMixture`。使用方式請自行參照使用手冊。另外，安裝方式稍不同於一般套件，Windows 版本為：`pip install -U scikit-learn`。請為圖二加上另一條由 `sklearn.mixture.GaussianMixture` 估計的混合常態 PDF 函數，如圖三。該套件使用著名的 EM 演算法估計相關參數。圖四則列出使用 `optimize.minimize` 與 `GaussianMixture` 估計的參數值。

```
import numpy as np
import scipy.optimize as opt
import matplotlib.pyplot as plt
from scipy.stats import norm, binom

# Weights for the two normal distributions
pi_1 = [0.3, 0.25, 0.7, 0.3, 0.7, 0.75] # Adjust these values as
needed

# Standard deviations for the two normal distributions in situation 1
sigma_1_sit1 = [10, 5, 2]
sigma_2_sit1 = [10, 5, 3]

# Standard deviations for the two normal distributions in situation 2
sigma_1_sit2 = [3, 2, 2] # Adjust these values as needed
sigma_2_sit2 = [2, 2, 2] # Adjust these values as needed

# Situation 1: mu_1 and mu_2 are close together
mu_1_sit1 = [12, 10, 12]
mu_2_sit1 = [20, 14, 13]
```

```

# Situation 2: mu_1 and mu_2 are far apart
mu_1_sit2 = [9, 13, 10]
mu_2_sit2 = [17, 20, 17]

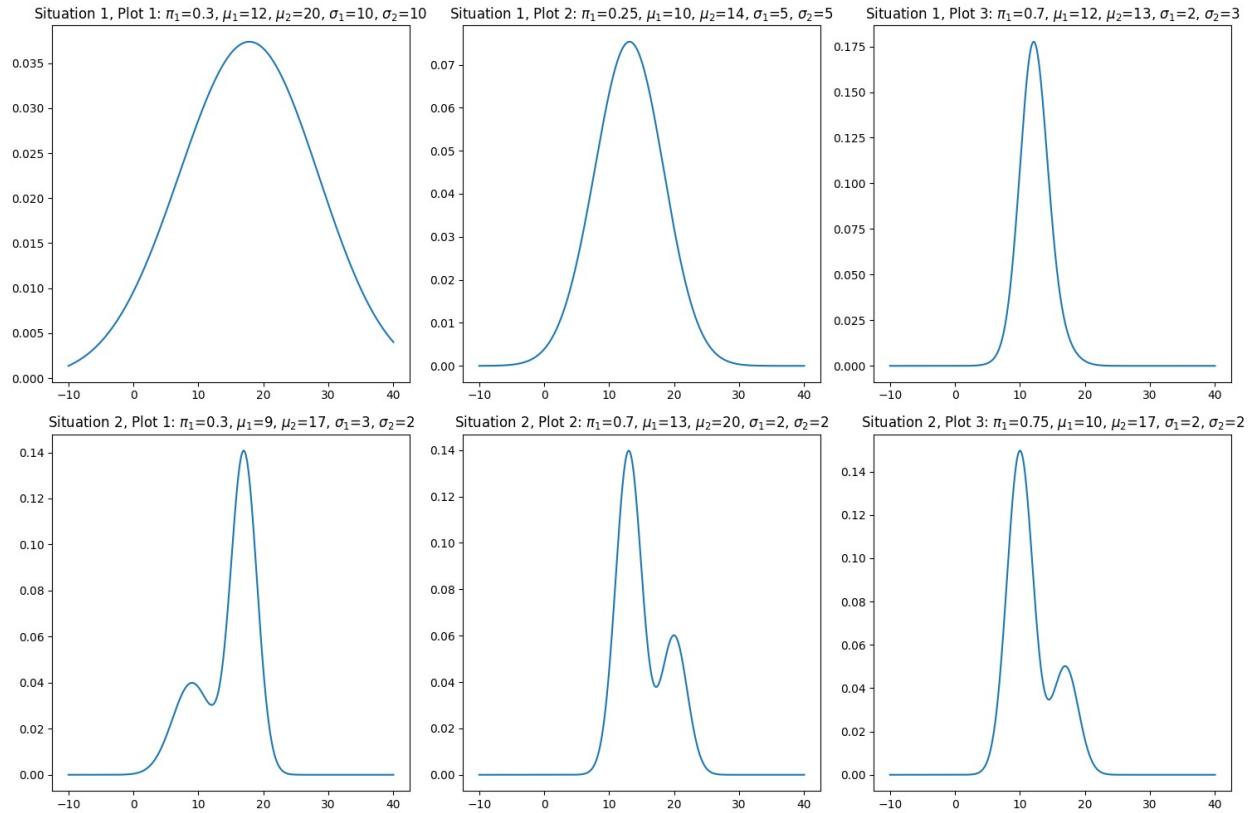
# Create subplots for both situations
fig, axs = plt.subplots(2, 3, figsize=(15, 10)) # 2 rows, 3 columns

# Plot for situation 1
for i in range(3):
    x = np.linspace(-10, 40, 1000)
    y = pi_1[i] * norm.pdf(x, mu_1_sit1[i], sigma_1_sit1[i]) + (1 -
pi_1[i]) * norm.pdf(x, mu_2_sit1[i], sigma_2_sit1[i])
    axs[0, i].plot(x, y)
    axs[0, i].set_title(f'Situation 1, Plot {i+1}:  $\pi_1 = \{pi\_1[i]\}$ ,
 $\mu_1 = \{mu\_1\_sit1[i]\}$ ,  $\mu_2 = \{mu\_2\_sit1[i]\}$ ,  $\sigma_1 = \{sigma\_1\_sit1[i]\}$ ,  $\sigma_2 = \{sigma\_2\_sit1[i]\}$ ')

# Plot for situation 2
for i in range(3, 6):
    x = np.linspace(-10, 40, 1000)
    y = pi_1[i] * norm.pdf(x, mu_1_sit2[i-3], sigma_1_sit2[i-3]) + (1
- pi_1[i]) * norm.pdf(x, mu_2_sit2[i-3], sigma_2_sit2[i-3])
    axs[1, i-3].plot(x, y)
    axs[1, i-3].set_title(f'Situation 2, Plot {i-2}:  $\pi_1 = \{pi\_1[i]\}$ ,  $\mu_1 = \{mu\_1\_sit2[i-3]\}$ ,  $\mu_2 = \{mu\_2\_sit2[i-3]\}$ ,
 $\sigma_1 = \{sigma\_1\_sit2[i-3]\}$ ,  $\sigma_2 = \{sigma\_2\_sit2[i-3]\}$ ')

plt.tight_layout()
plt.show()

```



```
import numpy as np
import scipy.optimize as opt
import matplotlib.pyplot as plt
from scipy.stats import norm, binom
from sklearn.mixture import GaussianMixture

# set up the parameters
pi1, mu1, sigma1, mu2, sigma2 = 0.7, 13, 2, 20, 2

# draw the mixture pdf
f = lambda x: pi1 * norm.pdf(x, mu1, sigma1) + (1-pi1) * norm.pdf(x,
mu2, sigma2)
x = np.linspace(-5, 30, 1000)

# the constraints, bounds and options
bnds = [(0, 1), (0, np.inf), (0, np.inf), (0, np.inf), (0, np.inf)]
opts = dict(displ = True, maxiter = 1e4)
x0 = [0.5, 10, 2, 20, 2] # initial guess

# list of sample sizes
sample_sizes = [50, 100, 300, 500, 1000, 10000]

# create subplots
fig, axs = plt.subplots(2, 3, figsize=(15, 10))
```

```

# ...
for i, n in enumerate(sample_sizes):
    # generate the simulated sample
    n1 = binom.rvs(n, p1)
    n2 = n - n1
    sample = np.r_[norm.rvs(mu1, sigma1, size = n1), norm.rvs(mu2,
sigma2, size = n2)]

    # max mle (min -mle)
    L = lambda x : -np.sum(np.log(x[0] * norm.pdf(sample, x[1], x[2])
+ (1 - x[0]) * norm.pdf(sample, x[3], x[4])))

    res = opt.minimize(L, x0 = x0, bounds = bnds, options = opts, tol
= 1e-8)

    # plot the true and estimated mixture pdf
    ax = axs[i//3, i%3]
    ax.plot(x, f(x), color = 'gold', linewidth = 3, label = 'True
mixture')

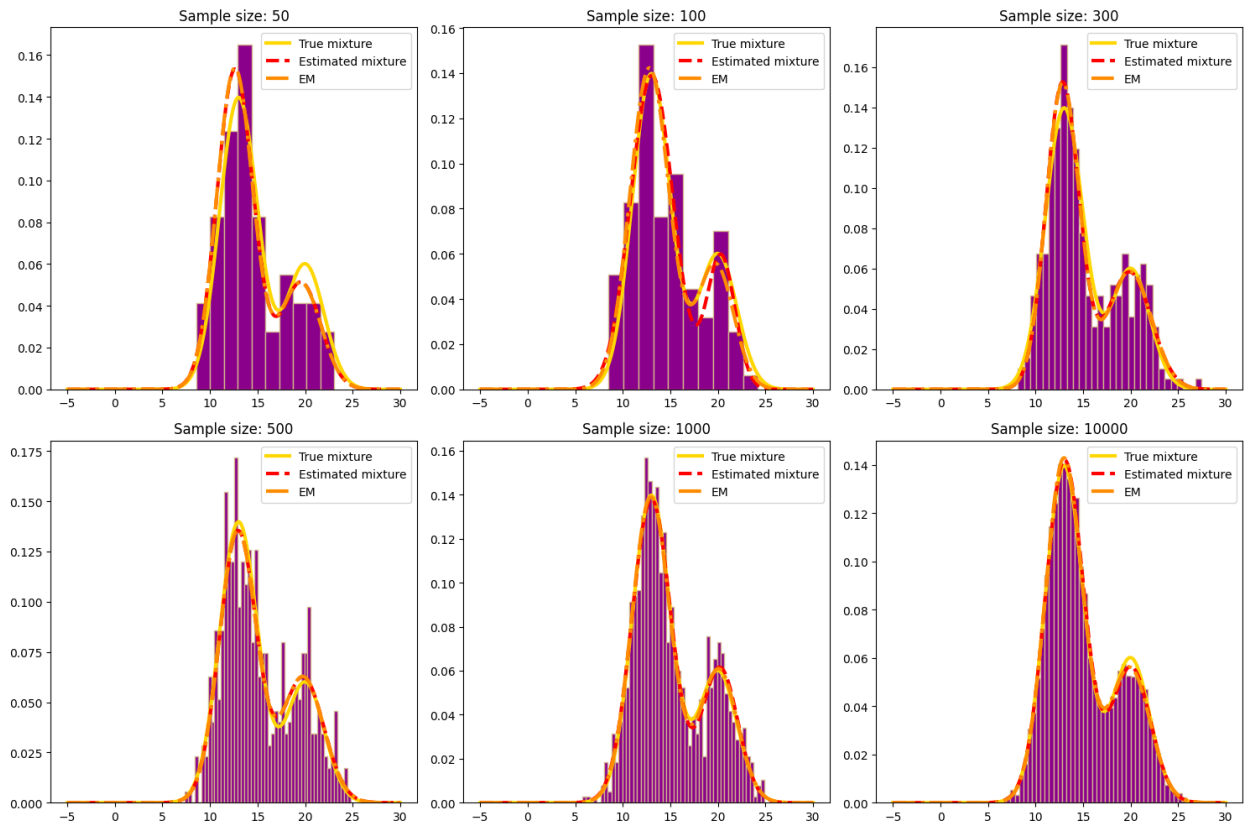
    #GaussianMixture
    gm = GaussianMixture(n_components=2)
    gm.fit(sample.reshape(-1, 1))
    gm_mixture = lambda x: np.exp(gm.score_samples(x.reshape(-1, 1)))

    # adjust the number of bins based on the sample size
    num_bins = min(max(n // 10, 10), 50)
    ax.hist(sample, num_bins, edgecolor = 'tan', density =
True, color="darkmagenta")

    f_hat = lambda x: res.x[0] * norm.pdf(x, res.x[1], res.x[2]) + (1-
res.x[0]) * norm.pdf(x, res.x[3], res.x[4])
    ax.plot(x, f_hat(x), color = 'red', linestyle = '--', linewidth =
3, label = 'Estimated mixture')
    ax.plot(x, gm_mixture(x), color='#FF8C00', linestyle='-.',
linewidth=3, label='EM')
    ax.legend()
    ax.set_title(f'Sample size: {n}')

plt.tight_layout()
plt.show()

```



```
import numpy as np
import scipy.optimize as opt
import matplotlib.pyplot as plt
from scipy.stats import norm, binom
from sklearn.mixture import GaussianMixture

# set up the parameters
pi1, mu1, sigma1, mu2, sigma2 = 0.3, 10, 5, 14, 5

# draw the mixture pdf
f = lambda x: pi1 * norm.pdf(x, mu1, sigma1) + (1-pi1) * norm.pdf(x,
mu2, sigma2)
x = np.linspace(-5, 30, 1000)

# the constraints, bounds and options
bnds = [(0, 1), (0, np.inf), (0, np.inf), (0, np.inf), (0, np.inf)]
opts = dict(dis = True, maxiter = 1e4)
x0 = [0.3, 11, 5, 14, 5] # initial guess

# list of sample sizes
sample_sizes = [50, 100, 300, 500, 1000, 10000]

# create subplots
fig, axs = plt.subplots(2, 3, figsize=(15, 10))
```

```

# ...
for i, n in enumerate(sample_sizes):
    # generate the simulated sample
    n1 = binom.rvs(n, p1)
    n2 = n - n1
    sample = np.r_[norm.rvs(mu1, sigma1, size = n1), norm.rvs(mu2,
sigma2, size = n2)]

    # max mle (min -mle)
    L = lambda x : -np.sum(np.log(x[0] * norm.pdf(sample, x[1], x[2])
+ (1 - x[0]) * norm.pdf(sample, x[3], x[4])))

    res = opt.minimize(L, x0 = x0, bounds = bnds, options = opts, tol
= 1e-8)

    # plot the true and estimated mixture pdf
    ax = axs[i//3, i%3]
    ax.plot(x, f(x), color = 'gold', linewidth = 3, label = 'True
mixture')

    #GaussianMixture
    gm = GaussianMixture(n_components=2)
    gm.fit(sample.reshape(-1, 1))
    gm_mixture = lambda x: np.exp(gm.score_samples(x.reshape(-1, 1)))

    # adjust the number of bins based on the sample size
    num_bins = min(max(n // 10, 10), 50)
    ax.hist(sample, num_bins, edgecolor = 'tan', density =
True, color="darkmagenta")

    f_hat = lambda x: res.x[0] * norm.pdf(x, res.x[1], res.x[2]) + (1-
res.x[0]) * norm.pdf(x, res.x[3], res.x[4])
    ax.plot(x, f_hat(x), color = 'red', linestyle = '--', linewidth =
3, label = 'Estimated mixture')
    ax.plot(x, gm_mixture(x), color='#FF8C00', linestyle='-.',
linewidth=3, label='EM')
    ax.legend()
    ax.set_title(f'Sample size: {n}')

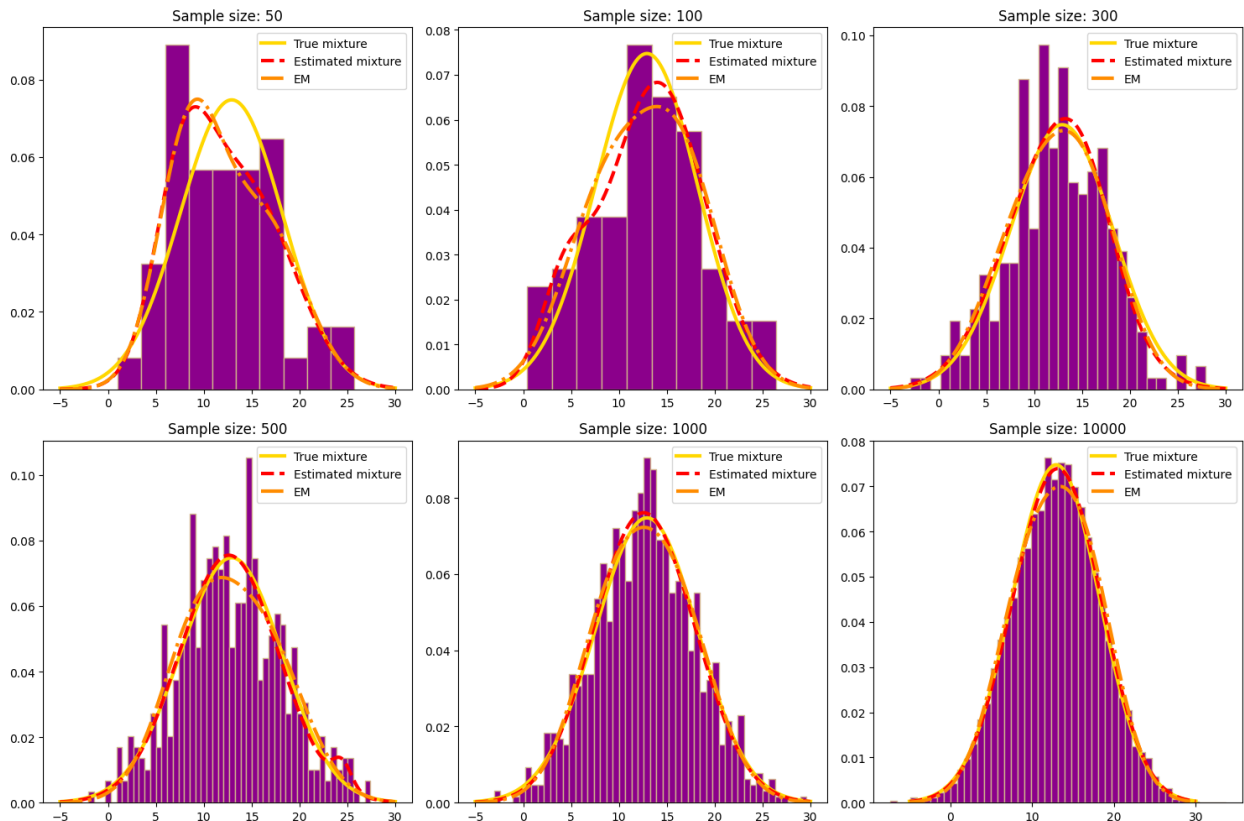
plt.tight_layout()
plt.show()

```

```

C:\Users\黃\AppData\Local\Packages\
PythonSoftwareFoundation.Python.3.11_qbz5n2kfra8p0\LocalCache\local-
packages\Python311\site-packages\scipy\stats\
_distn_infrastructure.py:1983: RuntimeWarning: divide by zero
encountered in divide
    x = np.asarray((x - loc)/scale, dtype=dtyp)

```



注意事項與討論:

- 這裡設定兩組不同參數讓同型呈現，在視覺上呈現一組有雙峰(兩組常態分佈)，另一組呈現單峰。
- 在利用估算法去跑 estimated mixture 時候，值得注意的是，如果今天用的 initial guess 的值差太多，那演算法就會算錯，我一開始就是用 μ 差太多導致整個曲線整個估計不好，就連樣本大(n)都會因為選錯參數而導致錯誤。
- 觀察圖片可以觀察看到在樣本小的時候 estimated mixture 和 EM 的估計會跟 TRUE 的有不同，但樣本大的時候，三者預估計幾乎重合。
- 整體而言，我會認為 estimated 的估計比 EM 好，圖形在樣本越變越大時，estimated 的線會與 TRUE 的更加重合。
- 雙峰的時候，可以發現 estimated mixture 和 EM 的估計 可以發現左峰估計超出更高，而右峰的估計較低。
- 整體而言，我會認為單峰樣本數在 300，但雙峰在 100 的時候整個估計會與 TRUE 的貼合達到不錯的效果。

研究二：限制式條件的最大值問題 Constraint optimization

計算下列最大概似估計 MLE 問題的參數 α, β ：

$$\max_{\alpha, \beta > 0} \ln L(\alpha, \beta)$$

其中的聯合概似函數為

$$L(\alpha, \beta) = \prod_{i=1}^n f_t(v_i \vee \alpha, \beta) F_T(u_i \vee \alpha, \beta)^{-1}$$

$$\text{where } f_t(v \vee \alpha, \beta) = \alpha \beta v^{\beta-1} \exp(-\alpha v^\beta)$$

$$F_T(u \vee \alpha, \beta) = 1 - \exp(-\alpha u^\beta)$$

變數 u, v 的 n 個樣本已知並存在檔案 UV.txt (點取下載)

建議依下列程序，逐步進行：（不熟悉的部分，再往前面的範例或章節找到可供參考的程式片段）

先下載資料檔，取出資料並觀察資料的樣子。

目標函數 $\ln L(\alpha, \beta)$ 需要進一步推導到比較適合的樣子，也就是將 $\prod$ 透過 $\ln$ 換成 $\sum$ 。並不是連乘的 $\prod$ 不能計算，非要換成連加的 $\sum$ 不可，而是當樣本數多時，連乘的計算比較不穩定，太大或太小的數值連乘可能超過硬體的極限。所以典型的最大概似估計問題，往往會在原目標函數前加上對數 $\ln$ 轉換成連加模式。請盡量將式子推到最精簡。（讀者仍可以試試看直接以 $\ln \prod$ 或 $\prod$ 的函數模式直接計算，也許也會得到相同的答案喔！）

利用推導到精簡的目標函數，繪製立體圖與等高線圖。繪圖時，需要摸索參數的範圍，找到最佳的觀察位置。畫得好，隱約可以看出最大值的位置（如下圖）。

接著開始部署 minimize 的各項停止條件及計算。有了等高線圖的幫助，通常答案已經呼之欲出，計算的結果只是得到一組更明確的數據。如圖中紅色的 X。

```
import numpy as np

# Use r before the string to treat \ as a normal character, not as an
# escape character
data_dir = r"C:\Users\黃\Desktop\新增資料夾 (2)\abba.txt"

# Load the data into a 2D NumPy array with utf-8 encoding
data = np.loadtxt(data_dir, encoding='utf-8')

ui_value, vi_value = data[:, 0], data[:, 1]

n=data.shape[0]

import numpy as np
import matplotlib.pyplot as plt

# Define the function L(alpha, beta)
def L(alpha, beta, vi_values, ui_values):
    return -np.sum(np.log(alpha * beta) + (beta - 1) *
np.log(vi_values) - alpha * vi_values**beta - np.log(1 - np.exp(-alpha
* ui_values**beta)), axis=-1)

# Generate a range of alpha and beta values
alpha_values = np.linspace(0.1, 2, 100)
beta_values = np.linspace(0.1, 3, 100)
```

```
# Create a meshgrid
Alpha, Beta = np.meshgrid(alpha_values, beta_values)

# Calculate the Z values for each (Alpha, Beta) pair
Z = L(Alpha[..., None], Beta[..., None], vi_value, ui_value)

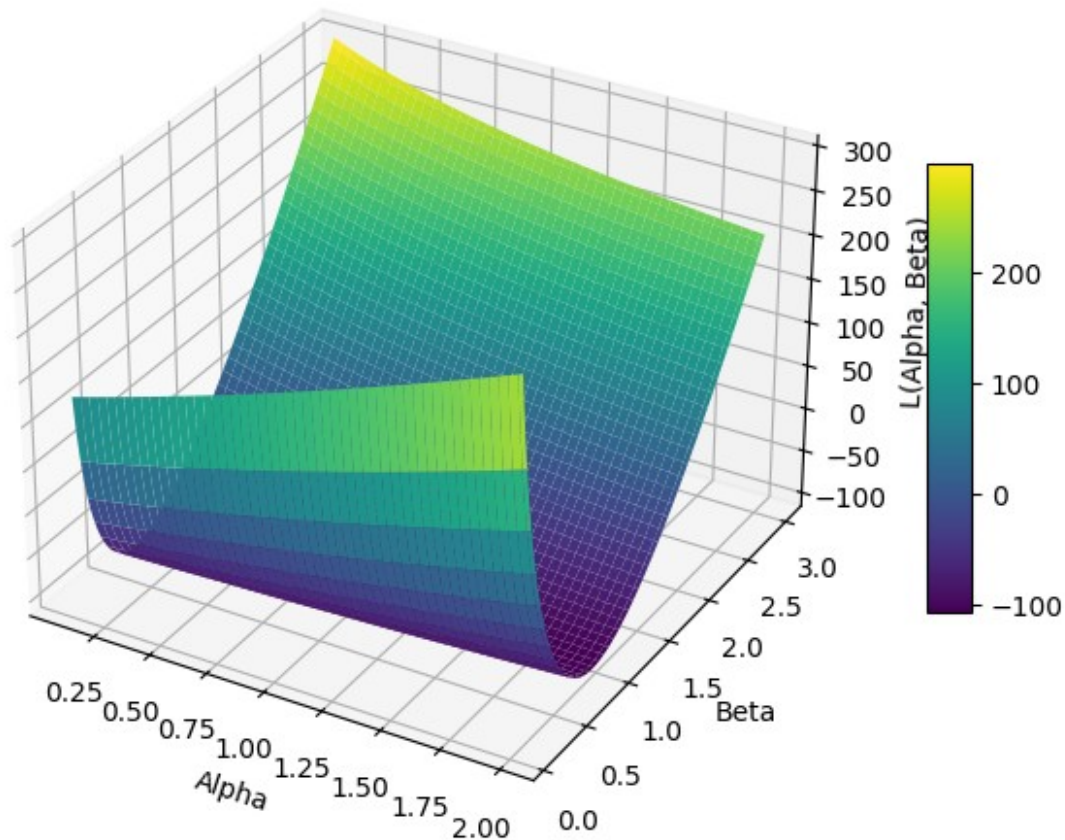
# Create a 3D plot
fig = plt.figure(figsize=(9, 6))
ax = plt.axes(projection='3d')

# Plot the surface
surf = ax.plot_surface(Alpha, Beta, Z, cmap='viridis')
fig.colorbar(surf, ax=ax, shrink=0.5, aspect=10)

# Set the labels and title
ax.set_xlabel('Alpha'), ax.set_ylabel('Beta'), ax.set_zlabel('L(Alpha, Beta)')
plt.title('Surface Plot of L(Alpha, Beta)')

plt.show()
```

Surface Plot of $L(\text{Alpha}, \text{Beta})$



```
import numpy as np
import matplotlib.pyplot as plt

# Define the function L(alpha, beta)
def L(alpha, beta, vi_values, ui_values):
    return -np.sum(np.log(alpha * beta) + (beta - 1) *
np.log(vi_values) - alpha * vi_values**beta - np.log(1 - np.exp(-alpha
* ui_values**beta))), axis=-1)

# Generate a range of alpha and beta values
alpha_values = np.linspace(0.5, 3, 100)
beta_values = np.linspace(0.5, 1.5, 100)

# Create a meshgrid
Alpha, Beta = np.meshgrid(alpha_values, beta_values)

# Calculate the Z values for each (Alpha, Beta) pair
Z = L(Alpha[..., None], Beta[..., None], vi_value, ui_value)

# Find the minimum point
```

```

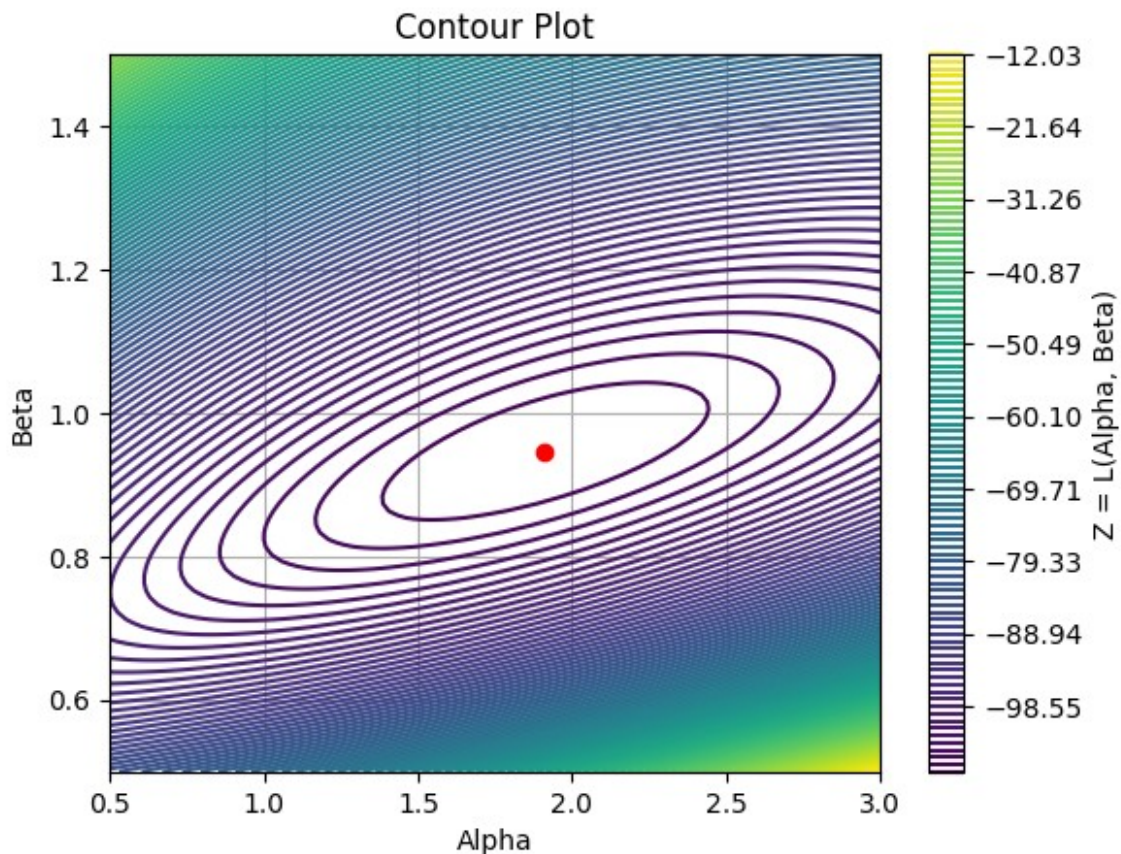
min_point = np.unravel_index(np.argmin(Z, axis=None), Z.shape)
min_alpha, min_beta = Alpha[min_point], Beta[min_point]

# To draw a contour plot
levels = np.linspace(Z.min(), Z.max(), 100) # levels of contour lines
contours = plt.contour(Alpha, Beta, Z, levels=levels) # check
dir(contours)

# Mark the minimum point
plt.plot(1.907349, 0.946438, 'ro') # 'ro' creates a red circle

cbar = plt.colorbar(contours)
plt.xlabel('Alpha'), plt.ylabel('Beta')
cbar.ax.set_ylabel('Z = L(Alpha, Beta)') # set colorbar label
plt.title('Contour Plot')
plt.grid(True)
plt.show()

```



注意事項與討論:

- 將函數加負號讓最值指令變成找最值。
- 利用立體圖可以去觀察整體分布，最小值大約坐落的位置，以及值的分布。
- 等高線圖標記出最小點後，就能很直觀的看出解答。

- 從圖片可以看出橢圓是一個凹槽。
- 下面打出確切的 alpha beta 值，然後再把 Minimum 的值精確打出
- 這裡基本上就是利用上課所教，配合影片中較困難運算 $-\text{np.sum}(\text{np.log}(\alpha * \beta) + (\beta - 1) * \text{np.log}(\text{vi_value}) - \alpha * \text{vi_value} * \beta - \text{np.log}(1 - \text{np.exp}(-\alpha * \text{ui_value} * \beta)))$, axis=-1) 解決此題。

```
from scipy.optimize import minimize

# Define the function L(alpha, beta)
def L(params):
    alpha, beta = params
    return -np.sum(np.log(alpha * beta) + (beta - 1) *
np.log(vi_value) - alpha * vi_value**beta - np.log(1 - np.exp(-alpha *
ui_value**beta)), axis=-1)

# Initial guess for alpha and beta
initial_guess = [2.2, 0.6]

# Call the minimize function
result = minimize(L, initial_guess, bounds=[(0.8, 2), (0.8, 2)])

# Print the result
print(f"The minimum value is at alpha = {result.x[0]}, beta =
{result.x[1]}")
print(f"The minimum value of the function is {result.fun}")

The minimum value is at alpha = 1.9073494275532852, beta =
0.9464388819868568
The minimum value of the function is -107.2085406964329
```